

Adaptive Graph Reweighting for Collaborative Filtering

Yijun Sheng
University of Macau
Macau SAR, China
yc17419@um.edu.mo

Ximing Chen
University of Macau
Macau SAR, China
yc37921@um.edu.mo

Pui Ieng Lei
University of Macau
Macau SAR, China
yc37460@um.edu.mo

Yanyan Liu
University of Macau
Macau SAR, China
yc07402@um.edu.mo

Zhiguo Gong*
University of Macau
Macau SAR, China
fstzgg@um.edu.mo

Abstract

Despite their success in Collaborative Filtering (CF), Graph Convolutional Networks (GCNs) are often viewed as Low-pass Graph Filters (LGFs) with task-specific supervision, while the influence of the underlying graph's spectral properties on LGF performance remains underexplored. Our analysis reveals that the performance of LGFs is strongly affected by algebraic connectivity. When connectivity is strong, LGFs tend to perform well; when it is weak, their effectiveness diminishes noticeably. This spectral sensitivity highlights an important limitation of existing models. To address this limitation, we propose *Graph Booster*, a learnable module that adaptively improves graph connectivity by reweighting edges. Unlike heuristic preprocessing, Graph Booster identifies bottleneck edges via spectral embeddings and adjusts their weights with a monotonic network guided by a lightweight graph connectivity regularizer. Integrated into LightGCN framework, our model **BoostGCN** achieves improvements over state-of-the-art methods, underscoring the significance of algebraic connectivity for graph-based CF.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Collaborative Filtering; Graph Reweighting; Matrix Perturbation Theory

ACM Reference Format:

Yijun Sheng, Ximing Chen, Pui Ieng Lei, Yanyan Liu, and Zhiguo Gong. 2026. Adaptive Graph Reweighting for Collaborative Filtering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792081>

1 Introduction

Collaborative Filtering (CF) [34, 35] is a widely adopted paradigm in recommender systems that infers users' preferences based on

their historical interactions with items. These user-item interactions can be naturally represented as bipartite graphs, enabling the effective use of Graph Convolutional Networks (GCNs) as powerful computational tools for recommendation tasks in recent years [2, 4, 18, 22, 26, 28, 43, 47, 49, 53–56]. Despite differences among these algorithms, they share a common technique: capturing high-order relationships through Low-pass Graph Filters (LGFs), which are crucial to their performance.

LightGCN [18] is widely regarded as a representative model among LGF-based approaches, offering both high performance and computational efficiency. Operating over the bipartite graph of user-item interactions, it applies multiple lightweight propagation layers, followed by optimization using the Bayesian Personalized Ranking (BPR) loss [33]. This mechanism embodies two key effects: (i) local smoothness via LGFs, where neighboring nodes become more similar through low-pass filtering [46, 57]; and (ii) preference alignment, where the BPR loss encourages interacting user-item pairs to be close in the embedding space. While the supervision mechanism via BPR has been extensively studied, the role of the graph's spectral structure in shaping LGFs performance remains comparatively underexplored. This raises a fundamental question: Which factors in the graph spectrum influence the effectiveness of LGFs for collaborative filtering?

We study this question through the lens of algebraic connectivity of the bipartite graph, which is measured as the second smallest eigenvalue λ_2 of the graph Laplacian. Intuitively, a larger λ_2 indicates fewer structural bottlenecks and stronger global connectivity of the graph, which facilitates information propagation. Our empirical experiments reveal that the LGF in LightGCN performs comparably to the vanilla LightGCN baseline on datasets with high algebraic connectivity, but degrades when connectivity is low. This observation aligns with prior findings that sharper spectral distribution benefits graph filtering [31]. These results highlight the potential of explicitly enhancing algebraic connectivity to improve the performance of LGF-driven collaborative filtering.

How can we increase λ_2 in a way that generalizes across datasets and remains efficient during training? Prior graph rewiring approaches [1, 21] typically rely on handcrafted heuristics applied as preprocessing steps. While computationally convenient, these manually defined heuristics lack adaptability to diverse graph topologies and to the evolving embedding geometry during learning. To overcome this limitation, we aim to integrate graph reweighting directly into the learning process, allowing the graph structure to be dynamically updated.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792081>

However, graph reweighting learning in the presence of noisy edges and outliers can be unstable. To stabilize optimization and ensure that graph reweighting effectively increases λ_2 , the model requires a regularization mechanism. Although the change in λ_2 induced by reweighting could serve this role, computing it exactly at every training step is prohibitively expensive. Thus, an efficient estimator is needed.

This leads to two key challenges: **(1) How to design a principled and trainable reweighting mechanism that explicitly increases algebraic connectivity?** **(2) How to efficiently estimate changes in algebraic connectivity during training to serve as a lightweight regularizer for stabilization?**

Addressing Challenge (1). We propose a spectral reweighting scheme designed to increase algebraic connectivity by identifying and reinforcing structural bottlenecks. Specifically, we compute Laplacian eigenmap embeddings for each node and assign additional weight to each observed edge proportional to the spectral distance between its endpoints. This augmentation is symmetric and globally informed, as eigenmap coordinates are derived from the Laplacian spectrum. Our design is grounded in *Spectral Graph Theory and Effective Resistance* [5, 10, 11], which suggest that edges bridging regions with large spectral separation tend to be bottlenecks, and strengthening them increases λ_2 . To enable flexible topology adjustment, we allow reweights to be either positive or negative, thereby strengthening or weakening edges as needed. Importantly, we constrain the magnitude of these reweights to remain below 1, ensuring the weighted adjacency matrix remains positive semi-definite throughout training.

Addressing Challenge (2). To address the high cost of repeatedly calculating algebraic connectivity, we introduce a matrix-perturbation-based estimator of the change in algebraic connectivity. Using *Matrix Perturbation Theory* [38], we approximate the first-order variation of λ_2 from sparse matrix-vector operations with the current Fiedler vector, avoiding repeated calls to expensive eigensolvers. Compared with the complexity of a normal eigensolver such as the `scipy.sparse.linalg.eigsh` cost of $O(t \cdot m)$ per update, our estimator runs in $O(r + n)$ time¹, enabling regularization without prohibitive overhead.

Putting these components together, we introduce *Graph Booster*, a learnable graph reweighting module that increases algebraic connectivity to improve LGFs performance. During training, we (i) learn additional per-edge weights from spectral distances and combine them with the original edge weights to form a weighted graph that evolves dynamically during training; (ii) incorporate the estimated increase in λ_2 as a regularization term in the training objective, encouraging reweighting steps that break through bottlenecks; and (iii) after each structural update, apply the standard LightGCN pipeline to extract collaborative signals. Although instantiated with LightGCN, the mechanism is model-agnostic for CF methods whose propagation acts as an LGF. To the best of our knowledge, explicitly targeting algebraic connectivity within an end-to-end learning framework remains largely unexplored in collaborative filtering. Our contributions can be summarized in three key aspects:

- We are the first to provide comprehensive theoretical and empirical analysis of algebraic connectivity in LGFs, establishing that increased algebraic connectivity leads to improved performance.
- We propose *Graph Booster*, a lightweight graph reweighting module that enhances LGFs by dynamically modifying the graph structure. The design integrates a reweighting mechanism grounded in spectral graph theory with a reweighting regularizer informed by matrix perturbation theory, enabling effective algebraic connectivity improvement.
- We conduct comprehensive experiments on four benchmark datasets, demonstrating that our method outperforms state-of-the-art CF models across multiple evaluation metrics.

2 Preliminary

2.1 Recall LightGCN

LightGCN [18], a representative GCN-based CF model that follows a LGF paradigm, serves as the foundation of our study. It focuses on embedding propagation over the user-item bipartite graph, where the neighbor aggregation process is as follows:

$$\begin{cases} \mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \mathbf{e}_i^{(k)}, \\ \mathbf{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \mathbf{e}_u^{(k)}, \end{cases} \quad (1)$$

where $\mathbf{e}_u$ and $\mathbf{e}_i$ denote the embedding of the user and item, $\mathcal{N}_u$ and $\mathcal{N}_i$ represent the neighbor set of node u and i respectively.

The final embeddings are computed as the average of the outputs from all layers:

$$\begin{cases} \tilde{\mathbf{e}}_u = \sum_{k=0}^K \alpha_k \mathbf{e}_u^{(k)}, \\ \tilde{\mathbf{e}}_i = \sum_{k=0}^K \alpha_k \mathbf{e}_i^{(k)}, \end{cases} \quad (2)$$

where K is the number of layers and $\alpha_k = \frac{1}{K+1}$. These two processes can be simplified into the following matrix form:

$$\tilde{\mathbf{E}} = \frac{1}{K+1} (\mathbf{I} + \tilde{\mathbf{A}} + \dots + \tilde{\mathbf{A}}^K) \mathbf{E}, \quad (3)$$

where $\tilde{\mathbf{A}}$ is the adjacent matrix of the bipartite graph. This implies that these two processes can essentially be viewed as a polynomial-weighted summation.

Given a training set $\mathcal{T}$, LightGCN is trained with the BPR loss and $\mathcal{L}_2$ regularization. For each triplet $(u, i^+, i^-) \in \mathcal{T}$ where i^+ and i^- denote the positive and negative items associated with u , respectively, the predicted score is the inner product of embeddings, $\hat{y}_{ui} = \tilde{\mathbf{e}}_u \cdot \tilde{\mathbf{e}}_i$, and the training objective is as follows:

$$\mathcal{L} = - \sum_{(u, i^+, i^-) \in \mathcal{T}} \ln \sigma(\hat{y}_{ui^+} - \hat{y}_{ui^-}) + \gamma \|\Theta\|_2, \quad (4)$$

where $\sigma(\cdot)$ is the sigmoid function and γ is the regularization coefficient. Intuitively, Eq. (1) aims to bring embeddings of connecting user and item closer, aligning with the objective in Eq. (4).

2.2 Graph Convolution in LightGCN Behaves as a Low-Pass Graph Filter

To identify key factors affecting graph convolution in LightGCN (Eqs. (1)-(2)), we begin our analysis within the *Graph Signal Processing* (GSP) [30] framework. Let $\tilde{\mathbf{L}} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ be the

¹ n nodes, m edges, r modified edges per update, and t eigensolver iterations (typically in the hundreds).

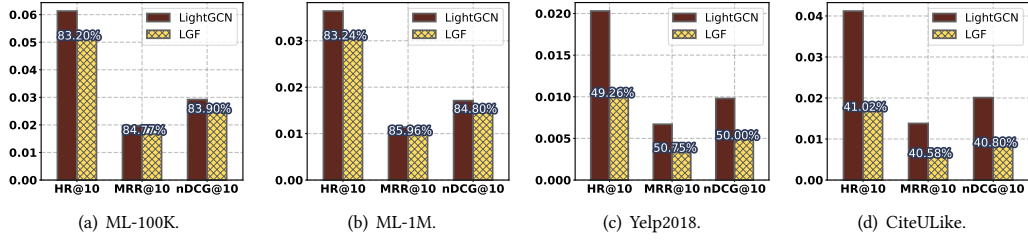


Figure 1: Performance comparison of LightGCN and LGF.

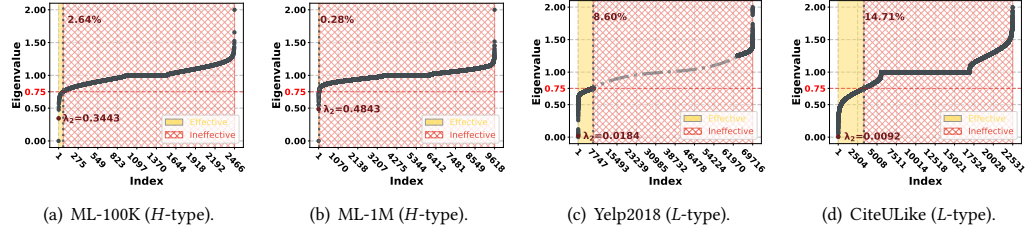


Figure 2: Eigenvalue distributions of different datasets. The yellow-shaded region highlights the range of effective signals ($\delta^* = 0.75$, results under other $\delta^* \in (0, 1)$ settings lead to consistent conclusions). For the large-scale Yelp dataset, only top 6,000 smallest and 6,000 largest eigenvalues are shown. The solid and dashed lines represent the observed and fitted data, respectively.

normalized Laplacian of a bipartite graph $\mathcal{G}$, where D is the degree matrix and A is the adjacency matrix. The eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_N = 2$ of $\tilde{L}$ constitute the spectrum of the Laplacian; together with their eigenvectors they define the graph spectral modes. The magnitude of λ_i equals the Dirichlet energy of the corresponding mode, so smaller λ_i indicate smoother, **lower-frequency** modes while larger λ_i indicate more oscillatory, **higher-frequency** modes. The normalized adjacency matrix $\tilde{A} = D^{-1/2}AD^{-1/2}$ has eigenvalues $\mu_i \in [-1, 1]$, with frequency defined as $\lambda_i = 1 - \mu_i$.

Then, we employ GSP to analyze the forward propagation of LightGCN as defined in Eq. (3). We perform the eigendecomposition of $\tilde{A}$ and substitute the result into Eq. (3), yielding this expression:

$$\tilde{A} = \tilde{V} \text{diag}(1 - \lambda_1, \dots, 1 - \lambda_N) \tilde{V}^T, \quad \tilde{V}^T \tilde{V} = I, \quad (5)$$

$$\tilde{E} = \tilde{V} \begin{bmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \frac{1}{K+1} \sum_{l=0}^K (1 - \lambda_N)^l = \frac{1 - (1 - \lambda_N)^{K+1}}{\lambda_N(K+1)} \end{bmatrix} \tilde{V}^T E. \quad (6)$$

Given a fixed value of $K \in \mathbb{Z}_+$, we observe that the monotonicity of the spectral filter coefficients $f(\lambda) = \frac{1 - (1 - \lambda)^{K+1}}{\lambda(K+1)}$ with respect to λ is characterized as follows (see Appendix 7.1 for details):

- K is odd: The function $f(\lambda)$ is strictly decreasing over $(0, 2)$.
- K is even: The function $f(\lambda)$ is strictly decreasing on $(0, \lambda^*)$ and strictly increasing on $(\lambda^*, 2)$, where $\lambda^* \in (1, 2)$ is the unique point satisfying $f'(\lambda^*) = 0$.

This observation suggests that the forward propagation of LightGCN behave as a Low-pass Graph Filter (LGF) over the spectral domain $\lambda \in [0, 1]$. Furthermore, as the number of layers K increases, the passband becomes progressively narrower. The low-pass characteristic of the graph convolution module adopted by LightGCN is not exclusive; in fact, many GCN-based models like LCFN [55] and LGCN [56], among others [28, 36] can be regarded as LGFs.

2.3 Algebraic Connectivity Is the Key

To understand the structural factors influencing the effectiveness of graph convolution, we compare the standard LightGCN model with its variant LGF across multiple benchmark datasets. LGF removes the training procedure, retaining only the forward propagation component of LightGCN (Eqs. (1)-(2)). Specifically, it applies multiple layers of light graph convolution to randomly initialized embeddings. This comparing design allows us to investigate impact of the low-pass graph filtering operation itself, and through this comparison, uncover the structural properties of the graph that influence the effectiveness LGFs.

2.3.1 Empirical Observations. By conducting some preliminary experiments, we make the following empirical observations:

(1) **Performance of LGF varies significantly across datasets.** As shown in Figure 1, LGF achieves performance comparable to LightGCN on ML-100K and ML-1M. However, on Yelp and CiteULike, it suffers a substantial performance drop.

(2) **Spectral characteristics align with performance patterns.** As shown in Figure 2, the ML-100K and ML-1M datasets exhibit steeper eigenvalue growth, reflecting high spectral concentration, which we denote as *H*-type. This characteristic is associated with stronger LGF performance. In contrast, Yelp and CiteULike display flatter spectral curves with lower spectrum growth (*L*-type), corresponding to notably weaker performance of LGF.

(3) **Algebraic connectivity is indicative of spectral type.** We further observe that the algebraic connectivity (i.e., the second smallest eigenvalue of the normalized Laplacian) correlates strongly with the *H*/*L*-type classification. Datasets with higher connectivity (e.g., ML-100K and ML-1M) exhibit *H*-type behavior, whereas those with lower connectivity (e.g., Yelp and CiteULike) fall into the *L*-type category.

2.3.2 Theoretical Discussion. We present a theoretical analysis to show how algebraic connectivity correlates with the performance of LGF. This analysis is based on two main insights:

- (1) The effectiveness of LGF depends on the concentration of informative collaborative signals.
- (2) Algebraic connectivity reflects the concentration of informative collaborative signals.

First, we provide a metric δ -**Low-Frequency Signal Ratio** to quantify the proportion of signals within $[0, \delta)$ frequency band.

Definition 2.1 (δ -Low-Frequency Signal Ratio). Let $\mathcal{G}$ be a bipartite graph with symmetric normalized Laplacian $\tilde{\mathbf{L}}$, and let $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_N = 2$ denote its eigenvalues. The δ -low-frequency signal ratio of $\mathcal{G}$ is defined as $R_\delta(\mathcal{G}) = \frac{K}{N}$, where N is the dimension of the matrix, K is the largest index of the eigenvalue such that $\lambda_K < \delta < 1$.

We now examine which frequency-band components carry informative signals for collaborative filtering, beginning with an analysis of how embedding disparities across nodes relate to the frequency characteristics of the underlying signals.

LEMMA 2.2. Let $\tilde{\mathbf{L}}$ denote the symmetric normalized Laplacian matrix of a user-item bipartite graph. After l layers of graph convolution, the squared Euclidean distance between the embeddings of two arbitrary users u and v is given by:

$$\|\mathbf{e}_u^{(l)} - \mathbf{e}_v^{(l)}\|^2 = \sum_{i=1}^n (\alpha_{u,i} - \alpha_{v,i})^2 (1 - \lambda_i)^{2l},$$

where λ_i is the i -th eigenvalue of $\tilde{\mathbf{L}}$, and $\alpha_{u,i}$, $\alpha_{v,i}$ are the projection coefficients of the embeddings $\mathbf{e}_u^{(0)}$ and $\mathbf{e}_v^{(0)}$ onto the eigenvector $\tilde{\mathbf{v}}_i$.

The detailed discussion is provided in Appendix 7.2. Based on Lemma 2.2, we derive the following corollary, which characterizes the role of mid-frequency components in distinguishing user embeddings.

COROLLARY 2.2.1 (MINOR CONTRIBUTION OF MID-FREQUENCY COMPONENTS FOR DISSIMILAR USERS). Let u and v be two users with distinct neighborhoods. For any number of propagation steps $l \geq 1$ and any arbitrarily small $\epsilon > 0$, there exists $\delta > 0$ such that the cumulative contribution from all mid-frequency components—those satisfying $|1 - \lambda_i| < \delta$ —is bounded by ϵ :

$$\sum_{i: |1 - \lambda_i| < \delta} (\alpha_{u,i} - \alpha_{v,i})^2 \cdot (1 - \lambda_i)^{2l} < \epsilon.$$

This implies that components with eigenvalues λ_i close to 1 offer limited discriminatory power between users with distinct neighborhoods.

Based on this corollary, we define the **Informative Signals** as those satisfying $\lambda < \delta^*$ (or $\lambda > 2 - \delta^*$), where $0 < \delta^* < 1$. Accordingly, we refer to $R_{\delta^*}(\mathcal{G})$ as the **Informative Signal Ratio**. As the eigenbasis of each user-item graph are dataset-specific, the distribution of informative signals varies accordingly. Based on this, we establish a connection between the informative signal ratio and the performance of LGF.

PROPOSITION 2.3 (LGF FAVORS DATASETS WITH LOWER INFORMATIVE SIGNAL RATIOS). Let $\mathcal{G}$ be a user-item interaction graph, and let $R_{\delta^*}(\mathcal{G})$ denote its informative signal ratio. Then, under a fixed convolution depth K , the performance of the LGF model is positively correlated with $\frac{1}{R_{\delta^*}(\mathcal{G})}$. In other words, LGF tends to perform better on datasets where the proportion of low-frequency components is relatively small.

PROOF. We begin by defining the **Passband Ratio** of LGF as the proportion of retained frequency components, given by $\eta = \frac{N^\dagger}{N}$, where $N^\dagger$ is the number of retained components and N is the total number of available components. From Eq. (6), it follows that for a fixed number of convolutional layers K , the passband ratio η remains constant in LGF; for instance, $\lim_{K \rightarrow \infty} \eta = \frac{1}{N}$.

Next, we define LGF's **Informative Signal Retention Ratio** as:

$$\rho = \frac{\min\{\eta, R_{\delta^*}(\mathcal{G})\}}{R_{\delta^*}(\mathcal{G})}.$$

This ratio quantifies the fraction of informative signals preserved by LGF. If $\eta \geq R_{\delta^*}(\mathcal{G})$, then LGF retains all informative components, yielding $\rho = 1$. Otherwise, LGF loses a fraction $1 - \rho$ of the informative signals, which leads to performance degradation.

Therefore, LGF performs better when $R_{\delta^*}(\mathcal{G})$ is small, as it increases the likelihood that all informative signals fall within the retained passband. $\square$

Remark. In summary, LGF performs optimally when the informative signal ratio $R_{\delta^*}(\mathcal{G})$ does not exceed the filter's passband ratio η , ensuring that all informative components are retained (i.e., $\rho = 1$). This condition is typically met in datasets where informative signals are highly concentrated near the spectral origin, such as in H -type graphs. In contrast, datasets with a broader or more uniform spectral distribution—referred to as L -type datasets—exhibit informative signals dispersed across a wider frequency range. In such cases, LGF filters out a substantial portion of useful information, resulting in degraded recommendation performance.

Next, we present a theoretical analysis demonstrating that algebraic connectivity can serve as an effective indicator of informative signals ratio.

THEOREM 2.4 (SPECTRAL CONCENTRATION THEOREM). Let $\mathcal{G}_A$ and $\mathcal{G}_B$ be two random bipartite graphs with a degree sequence satisfying $d_{\min} \gg \sqrt{\tilde{d}}$, where $\tilde{d}$ denotes the average vertex degree (i.e. $\tilde{d} = \frac{1}{|V|} \sum_{v \in V} d_v$). Then for any $\delta \in (0, 1)$, $\lambda_2(\mathcal{G}_A) > \lambda_2(\mathcal{G}_B) \Rightarrow R_\delta(\mathcal{G}_A) < R_\delta(\mathcal{G}_B)$.

Theorem 2.4 establishes a spectral link between algebraic connectivity and the concentration of low-frequency signals. The proof can be found in Appendix 7.3. Building on this result, we obtain the following corollary:

COROLLARY 2.4.1. The algebraic connectivity $\lambda_2(\mathcal{G})$ inversely correlates with informative signal ratio: $\lambda_2(\mathcal{G}) \uparrow \Rightarrow R_{\delta^*}(\mathcal{G}) \downarrow \forall \delta^* \in (0, 1)$, indicating that higher λ_2 implies stronger informative signal concentration.

Since algebraic connectivity reflects the degree to which informative signals concentrate in the low-frequency spectrum, and LGF performance depends on such concentration, it serves as a reliable structural indicator for LGF effectiveness.

2.3.3 Summary. Based on the discussion, we can enhance the algebraic connectivity of the graph through graph reweighting, thereby improving model performance. From a spectral perspective, increasing algebraic connectivity compresses more useful signal components into LGF's passband, allowing informative frequencies to be better preserved during propagation. From a spatial perspective,

user-item graphs often contain both well-connected (H -type) and sparsely connected (L -type) subgraphs, leading to diverse structural patterns. This heterogeneity causes different nodes to favor different signal bandwidths: passing more frequency components benefits L -type subgraphs by alleviating over-squashing, but leads to over-smoothing in H -type subgraphs. Conversely, a narrow pass-band preserves sharp distinctions in H -type areas but compresses signals in L -type subgraphs. Consequently, a fixed convolution depth K cannot optimally serve all nodes. Improving algebraic connectivity helps flatten these structural disparities, reducing node sensitivity to K and enhancing performance.

3 Methodology

3.1 Problem Formulation

Let $\mathcal{U} = \{u_1, \dots, u_{|\mathcal{U}|}\}$ and $\mathcal{I} = \{i_1, \dots, i_{|\mathcal{I}|}\}$ denote the user and item sets. Each user u has an interaction history $R_u = [i_{u,1}, \dots, i_{u,|R_u|}]$, from which we construct a binary interaction matrix $\mathbf{R} \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}$, where $y(u, i) = 1$ if $i \in R_u$, and 0 otherwise. Based on $\mathbf{R}$, we build a user-item bipartite graph $\mathcal{G}$, where an undirected edge connects u and i if $y(u, i) = 1$. The normalized Laplacian and adjacency matrices of $\mathcal{G}$ are denoted as $\tilde{\mathbf{L}}$ and $\tilde{\mathbf{A}}$, respectively. We obtain the initial algebraic connectivity λ_2 via eigendecomposition $\tilde{\mathbf{L}} = \tilde{\mathbf{V}}\tilde{\mathbf{\Lambda}}\tilde{\mathbf{V}}^\top$, where $\tilde{\mathbf{\Lambda}} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_{|\mathcal{U}|+|\mathcal{I}|})$. Initial node embeddings are denoted by $\mathbf{E} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|) \times h}$, where h is the embedding dimension.

Our goal is to predict which items in $\mathcal{I}$ a user u is likely to interact with. Formally, we aim to learn the interaction probability $\hat{y}(u, i) = \mathcal{F}(u, i | \mathbf{R})$. Unlike conventional GCN-based methods, we seek to improve recommendation quality by reweighting the graph to increase its algebraic connectivity.

3.2 Graph Booster

A simple approach is to modify the graph during preprocessing by estimating each edge's effect on algebraic connectivity. Prior work [3, 21] provides efficient approximations that avoid repeated eigendecomposition. However, such fixed heuristics are graph-agnostic and often fail to generalize across diverse topologies. To address this, we integrate reweighting into a learnable framework, enabling dynamic adaptation to graph structure.

3.2.1 Bottleneck Score Learning. Based on the *Spectral Graph Theory* [10], a graph's algebraic connectivity is determined by the presence of bottleneck edges. Identifying such edges and reweighting them can therefore enhance the algebraic connectivity.

Let $\{\tilde{\mathbf{v}}_i\}_{i=2}^{k+1}$ be the first k nontrivial eigenvectors of $\tilde{\mathbf{L}}$. The Fiedler vector $\tilde{\mathbf{v}}_2$ is directly related to algebraic connectivity: edges whose endpoints differ significantly along this vector are more likely to be bottlenecks [13]. To capture richer structural information, we leverage additional eigenvectors and first construct an eigenmap:

$$\mathbf{U} = [\tilde{\mathbf{v}}_2, \tilde{\mathbf{v}}_3, \dots, \tilde{\mathbf{v}}_{k+1}], \quad (7)$$

where $\mathbf{U} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|) \times k}$. The difference between an edge's endpoints in this space provides a natural indicator of its bottleneck potential [25]. We then learn the bottleneck score for edge (i, j) based on their separation in the eigenmap space:

$$\mathbf{B}[i, j] = g(|\mathbf{U}[i, :] - \mathbf{U}[j, :]|), \quad (8)$$

where $\mathbf{U}[i, :], \mathbf{U}[j, :] \in \mathbb{R}^k$ denote the rows of the eigenvector matrix $\mathbf{U}$ corresponding to nodes i and j . The function $g(\cdot)$ is designed to model the importance of different spectral dimensions.

We further design $g(\cdot)$ as a **Monotonic Network** [37] that is monotonically increasing with respect to its input, ensuring that edges with greater endpoint differences are more likely to be bottlenecks [10] and thus necessitating the addition of weights. Technically, we enforce monotonicity by applying an *ReLU* activation function to the trainable weights, i.e., $\phi = \text{ReLU}(\phi_{\text{init}})$, which ensures that all layer weights remain strictly non-negative:

$$\mathbf{W}^{(i)} = \text{ReLU}(\mathbf{W}_{\text{init}}^{(i)}), \quad \text{for } i = 1, \dots, L, \quad (9)$$

$$\mathbf{h}_i = f_i(\mathbf{h}_{i-1}; \mathbf{W}^{(i)}), \quad \text{for } i = 1, \dots, L-1, \quad (10)$$

$$\mathbf{y} = f_L(\mathbf{h}_{L-1}; \mathbf{W}^{(L)}), \quad (11)$$

where f_i represents the linear transformation of the i -th layer with *Tanh* activation, $\mathbf{h}_i$ is the output of the i -th hidden layer ($\mathbf{h}_0 = |\mathbf{U}[i, :] - \mathbf{U}[j, :]|$), and L denotes the total number of layers. In practice, we adopt a lightweight three-layer configuration.

As eigenvalues increase, the corresponding eigenvectors become less effective at capturing global bottleneck edges [25]. Therefore, we impose a weight-decreasing initialization on the monotonic network parameters, with the weight of the first dimension being the largest and decreasing along higher dimensions:

$$\mathbf{W} = \begin{bmatrix} w_{11} & \dots & w_{1d} \\ \vdots & \ddots & \vdots \\ w_{k1} & \dots & w_{kd} \end{bmatrix}, \quad w_{1*} \geq w_{2*} \geq \dots \geq w_{k*} \geq 0. \quad (12)$$

This design ensures that the network output increases monotonically with the input differences while preserving the decreasing importance of higher spectral dimensions. We summarize the process as $\mathbf{B}[i, j] = \text{MonoNet}(|\mathbf{U}_i - \mathbf{U}_j|)$.

To ensure robust bottleneck score computation, we mitigate extreme outliers via percentile-based clipping followed by normalization. Specifically, values are clipped to the $[0.1, 99.9]$ percentile range, centered by subtracting the mean, rescaled by the range, and finally transformed with *tanh*:

$$\mathbf{B}'[i, j] = \tanh\left(\frac{\text{clip}(\mathbf{B}[i, j], \ell, \tau) - \bar{\mu}}{\tau - \ell + \varepsilon}\right), \quad (13)$$

where ℓ and τ denote the 0.1th and 99.9th percentiles, and $\bar{\mu}$ is the mean of the clipped values. This design allows for both positive and negative scores while ensuring robustness.

3.2.2 Graph Reweighting. After computing the bottleneck score for each edge, we enhance the original edge weights with these scores to strengthen graph connectivity. Intuitively, edges with higher bottleneck scores play a more critical role in global connectivity and therefore need larger weights. Thus, we update the adjacency matrix by incorporating bottleneck scores on existing edges. Among possible strategies, we adopt a simple additive update, followed by a β -parameterized symmetric normalization inspired by [31]:

$$\mathbf{A}'[i, j] = (\mathbf{A}[i, j] + \gamma_1 \mathbf{B}'[i, j]) \cdot \mathbf{1}_{\{\mathbf{A}[i, j] \neq 0\}}, \quad (14)$$

$$\tilde{\mathbf{A}}' = \mathbf{D}'^{-\beta} \mathbf{A}' \mathbf{D}'^{-\beta}, \quad \text{where } \mathbf{D}'_{ii} = \sum_j \mathbf{A}'_{ij}. \quad (15)$$

We initialize β to 0.5 and treat it as a trainable parameter. A smaller β typically concentrates the spectrum [31]. The coefficient $\gamma_1 \in (0, 1]$ controls the augmentation strength and is constrained to preserve the positive semi-definiteness of the updated adjacency matrix.

3.2.3 Lightweight Regularizer. The reweighting process yields an enhanced adjacency matrix $\tilde{\mathbf{A}}$. During the graph reweighting process, the presence of noisy edges and outliers can be unstable. To systematically address this, we introduce a principled method to ensure the effectiveness of graph reweighting and stabilizes optimization. Leveraging *Matrix Perturbation Theory* [38], we develop a generalized first-order eigenvalue perturbation approximation that extends existing **single-edge** modifications approximation [3, 21] to handle changes in **multiple** edge weights.

$$\Delta\lambda_2 \approx (1 - \lambda_2)\tilde{\mathbf{v}}_2^\top \mathbf{D}^{-1} \Delta_{\mathbf{D}} \tilde{\mathbf{v}}_2 - \tilde{\mathbf{v}}_2^\top \mathbf{D}^{-\frac{1}{2}} \Delta_{\mathbf{A}} \mathbf{D}^{-\frac{1}{2}} \tilde{\mathbf{v}}_2, \quad (16)$$

$$\Delta_{\mathbf{A}} = \mathbf{A}' - \mathbf{A}, \quad (17)$$

$$\Delta_{\mathbf{D}} = \text{diag}(\sum \Delta_{\mathbf{A}}[0, :], \dots, \sum \Delta_{\mathbf{A}}[n, :]), \quad (18)$$

where $\tilde{\mathbf{v}}_2$ denotes the corresponding eigenvector of λ_2 . The detailed deduction can be found in Appendix 7.4. Since each reweighting may involve both the increase and decrease of edge weights simultaneously, we simply assume that the changes in $\Delta_{\mathbf{D}}$ are sufficiently small ($\Delta_{\mathbf{D}} \approx 0$) to reduce computational complexity. Based on this assumption, we propose the following estimation formula:

$$\Delta\lambda_2 \approx -\tilde{\mathbf{v}}_2^\top \mathbf{D}^{-\frac{1}{2}} \Delta_{\mathbf{A}} \mathbf{D}^{-\frac{1}{2}} \tilde{\mathbf{v}}_2. \quad (19)$$

By simultaneously accounting for changes in multiple edge weights, our approach provides a more comprehensive assessment of their impact on the graph's algebraic connectivity during reweighting. Computing eigenvalue via `eigsh` requires $\mathcal{O}(t \cdot m)$ time, whereas our approximation can be obtained in $\mathcal{O}(r + n)$ time, yielding substantial computational savings. This low-cost approximation further enables the change in $\Delta\lambda_2$ to be seamlessly incorporated as an lightweight regularizer.

3.3 Low-pass Graph Filtering

After obtaining the reweighted graph with adjacency $\tilde{\mathbf{A}}'$, we directly apply existing LGFs as the downstream model:

$$\tilde{\mathbf{E}} = \text{LGF}(\tilde{\mathbf{A}}, \mathbf{E}), \quad (20)$$

where $\mathbf{E}$ are initial node embeddings. In practice, $\text{LGF}(\cdot)$ can correspond to well-established architectures such as LightGCN [18] or NGCF [43]. In our BoostGCN model, we use LightGCN as the $\text{LGF}(\cdot)$. This formulation allows our reweighted graph to seamlessly integrate with existing collaborative filtering models.

3.4 Prediction and Loss

To predict user-item interactions, we first extract the user embedding $\tilde{\mathbf{e}}_u$ and item embedding $\tilde{\mathbf{e}}_i$ from the embedding matrix $\tilde{\mathbf{E}}$. The interaction score is calculated by taking the inner product of these two embeddings:

$$\hat{y}(u, i) = \tilde{\mathbf{e}}_u \cdot \tilde{\mathbf{e}}_i^\top. \quad (21)$$

To optimize the parameters of BoostGCN, we adopt the Bayesian Personalized Ranking (BPR) loss [33], supplemented by the $\Delta\lambda_2$

regularizer to stabilize learning and L_2 regularization loss to prevent overfitting. Given the training data $\mathcal{T}$, the overall objective is defined as:

$$\begin{aligned} \mathcal{L}_{\text{BPR}} &= \sum_{(u, i^+, i^-) \in \mathcal{T}} -\ln(\sigma(\hat{y}_{ui^+} - \hat{y}_{ui^-})), \\ \mathcal{L} &= \mathcal{L}_{\text{BPR}} - \gamma_2 \Delta\lambda_2 + \gamma_3 L_2, \end{aligned} \quad (22)$$

where i^+ denotes an item interacted with by user u and i^- represents an item not interacted with by u . The hyperparameters $\{\gamma_2, \gamma_3\}$ control the balance between the three loss components.

3.5 Discussion

In contrast to attention-based models such as GAT [40], which learn neighborhood-specific weights via an attention mechanism followed by softmax normalization, our Graph Booster derives edges' bottleneck scores from a global eigenmap of the graph Laplacian. This yields several fundamental distinctions. **(i) locality vs. globality:** GAT attention is locally normalized and thus relative within each neighborhood; bottleneck scores are absolute, capturing global structural information into every edge. **(ii) symmetry vs. asymmetry:** bottleneck scores based on $|\mathbf{U}[i, :] - \mathbf{U}[j, :]|$ are inherently symmetric, naturally reflecting edge importance in graphs; GAT attention is asymmetric, hindering a globally consistent bottleneck ranking. **(iii) normalization and competition:** $\text{softmax}(\cdot)$ enforces neighbor-wise competition, making attention values inherently relative; bottleneck scores quantify intrinsic structural significance independent of local normalization. **(iv) interpretability and theoretical guarantees:** spectral reweighting aligns with algebraic connectivity via the Spectral Graph Theory [10] and Effective Resistance [5, 11], providing a provable basis for identifying structural bottlenecks; GAT attention lacks explicit spectral guarantees. **(v) prior bias and robustness:** Our design enforces monotonic, dimension-wise decreasing weights, explicitly encoding a prior that emphasizes lower-order eigenvectors, whereas GAT must implicitly learn such an inductive bias from data.

4 Experiments

4.1 Experimental Settings

4.1.1 Datasets. We evaluate our model on four widely used real-world benchmark datasets: *MovieLens-100K*² (ML-100K), *MovieLens-1M*² (ML-1M) [16], *Yelp2018*³, and *CiteULike*⁴ [41]. Table 1 summarizes their key statistics, where λ_2 denotes the algebraic connectivity of the corresponding user-item bipartite graph.

Table 1: Statistics of datasets after preprocessing.

Dataset	ML-100K	ML-1M	Yelp2018	CiteULike
Interaction #	82,460	836,421	1,561,406	210,537
User #	943	6,038	31,668	5,551
Item #	1,523	3,580	38,048	16,980
Density	0.0574	0.0387	0.0013	0.0022
λ_2	0.3442	0.4843	0.0184	0.0092 [‡]

[‡] $\lambda_2 \approx 0$ suggests near-disconnectedness of the graph.

²<https://grouplens.org/datasets/movielens/>

³<https://www.yelp.com/dataset>

⁴<https://github.com/js05212/citeulike-a>

Table 2: Overall recommendation performance comparison. The Avg. Rank represents the ranking of the average ranking of individual metrics across all models. Except for Avg. Rank, all other metrics indicate better performance with higher values.

Dataset	ML-100K			ML-1M			Yelp2018			CiteULike			Avg. Rank
Model	HR@10	MRR@10	nDCG@10	HR@10	MRR@10	nDCG@10	HR@10	MRR@10	nDCG@10	HR@10	MRR@10	nDCG@10	
NGCF	0.0594	0.0178	0.0273	0.0395	0.0120	0.0183	0.0263	0.0087	0.0128	0.0337	0.0104	0.0157	6.6
LightGCN	0.0613	<u>0.0197</u>	0.0292	0.0364	0.0114	0.0171	0.0203	0.0067	0.0098	0.0412	0.0138	0.0201	6.3
UltraGCN	0.0500	0.0155	0.0234	0.0317	0.0090	0.0142	0.0069	0.0021	0.0032	0.0171	0.0056	0.0082	12.4
IMP-GCN	0.0498	0.0151	0.0231	0.0316	0.0092	0.0144	0.0348	0.0106	0.0162	0.0541	0.0162	0.0249	7.25
LCFN	0.0484	0.0147	0.0225	0.0318	0.0099	0.0150	0.0068	0.0023	0.0033	0.0163	0.0054	0.0079	12.9
GDE	0.0544	0.0178	0.0262	0.0320	0.0101	0.0152	0.0230	0.0075	0.0111	0.0301	0.0105	0.0150	8.4
LGCN	0.0656	0.0197	0.0302	0.0379	0.0123	0.0182	0.0160	0.0053	0.0078	0.0329	0.0109	0.0159	7.0
PGSP	0.0090	0.0022	0.0038	0.0246	0.0064	0.0106	OOM	OOM	OOM	0.0415	0.0107	0.0177	14.0
SGFCF	0.0291	0.0066	0.0117	0.0236	0.0049	0.0091	0.0043	0.0008	0.0015	0.0103	0.0015	0.0034	16.0
SGL	0.0434	0.0122	0.0193	0.0304	0.0090	0.0139	0.0177	0.0051	0.0080	0.0476	0.0119	0.0199	11.1
SSL4Rec	0.0184	0.0051	0.0081	0.0254	0.0076	0.0117	0.0161	0.0048	0.0074	0.0287	0.0120	0.0120	13.8
SimGCL	0.0542	0.0158	0.0245	0.0294	0.0082	0.0130	0.0360	<u>0.0114</u>	0.0170	0.0483	0.0122	0.0203	7.4
XSimGCL	0.0211	0.0053	0.0089	0.0369	0.0105	0.0165	0.0374	<u>0.0114</u>	<u>0.0174</u>	0.0513	0.0141	0.0226	6.8
LT-OCF	0.0489	0.0151	0.0229	0.0343	0.0122	0.0172	0.0343	0.0113	0.0166	<u>0.0574</u>	<u>0.0183</u>	<u>0.0272</u>	5.6
BSPM	0.0535	0.0172	0.0255	0.0323	0.0102	0.0152	0.0209	0.0070	0.0102	<u>0.0165</u> [‡]	<u>0.0038</u> [‡]	<u>0.0066</u> [‡]	7.8
CDE-CF	0.0510	0.0166	0.0245	<u>0.0403</u>	<u>0.0128</u>	<u>0.0191</u>	0.0297	0.0097	0.0143	0.0409	0.0135	0.0198	5.8
BoostGCN	0.0692	0.0212	0.0320	0.0451	0.0138	0.0210	<u>0.0368</u>	0.0121	0.0178	0.0587	0.0186	0.0278	1.1
Improve (%)	5.49	7.61	6.62	11.91	7.81	9.95	-1.68	5.74	2.29	2.26	1.64	2.21	

[‡] Original setting gave invalid results; we report scores under our settings and exclude this metric from ranking.

4.1.2 Evaluation Metrics. We adopt three commonly used evaluation metrics to assess recommendation performance: *Hit Ratio at k* ($HR@k$), *Mean Reciprocal Rank at k* ($MRR@k$), and *normalized Discounted Cumulative Gain at k* ($nDCG@k$). The HR metric quantifies the binary accuracy by determining whether recommended items have been interacted with by users. Both MRR and $nDCG$ measure the quality of the ranking, reflecting how well the relevant items are positioned within the recommendation list. In our experiments, all metrics are calculated using the top-10 recommended items ($k = 10$), with higher scores indicating superior model effectiveness.

4.1.3 Baselines. To validate the effectiveness of our proposed model, we conduct comparisons with several state-of-the-art approaches categorized as follows: *Spatial-domain graph-based models* that exploit the spatial relationships among nodes, including NGCF [43], LightGCN [18], UltraGCN [28], and IMP-GCN [26]; *Spectral-domain graph-based models* which utilize graph signal processing techniques to better analyze graph-structured data, such as LCFN [55], GDE [32], LGCN [56], PGSP [27], and SGFCF [31]; *Contrastive learning-based models* that learn robust representations by distinguishing positive from negative samples, including SGL [47], SSL4Rec [51], SimGCL [54], and XSimGCL [53]; and *ODE-based models* which leverage continuous dynamics of ordinary differential equations for flexible and stable information propagation, exemplified by LT-OCF [8], BSPM [7], and CDE-CF [50].

4.1.4 Settings. BoostGCN⁵ is developed with TensorFlow. We adopt the Nadam optimizer uniformly across all experiments and keep hyperparameters fixed unless otherwise specified in their codes. The default settings include an embedding dimension of 64, an initial learning rate of 1×10^{-3} , and a batch size of 2,048. For the MovieLens datasets, rating matrices are binarized using a threshold of 2. To facilitate rigorous evaluation, data splits follow a 7:1:2 ratio for training, validation, and testing, respectively. Each experiment is repeated at least five times, and the reported results represent the average metrics. Consistent with prior studies [18, 43], we adopt a training scheme that pairs one positive item with one negative

item and apply an all-ranking protocol for testing, focusing primarily on ranking performance. Early stopping⁶ is employed to avoid overfitting and reduce the complexity of hyperparameter tuning.

4.2 Overall Performance

Table 2 reports the overall results, where the best-performing model is highlighted in bold and the second-best is underlined; reported improvements are relative to the second best. BoostGCN yields consistent gains across datasets, with average relative gains of 6.75%, 9.89%, 2.12%, and 2.04% on ML-100K, ML-1M, Yelp2018, and CiteULike, respectively. These improvements can be attributed to its adaptive graph reweighting mechanism guided by algebraic connectivity: by dynamically evaluating and adjusting adjacency weights during training, BoostGCN effectively mitigates noise, adapts to distributional shifts, and preserves structural semantics, thereby enhancing information propagation and representation quality. Notably, models grounded in Ordinary Differential Equations (ODEs)—including LT-OCF, BSPM, and CDE-CF—also demonstrate strong performance. By modeling message passing as continuous dynamics, these approaches enable finer-grained control over information flow and alleviate oversmoothing, which collectively improves both expressiveness and empirical performance.

4.3 Study on Graph Booster

4.3.1 Can We Use Simple Preprocessing? We compare our end-to-end learning strategy with a static variant termed **EA+LightGCN**, where 5,000 edges are added in preprocessing according to Eq. (16) to increase algebraic connectivity, and standard LightGCN training is subsequently applied. As shown in Table 3, this preprocessing improves connectivity but often degrades recommendation accuracy, indicating that optimizing connectivity in isolation may distort structural semantics. In contrast, our graph reweighting module adaptively enhances connectivity during training, consistently yielding larger algebraic connectivity, particularly on sparsely connected graphs, and translating these gains into improved downstream performance.

⁵The source code is accessible at <https://github.com/jaxyijunsheng/BoostGCN>

⁶Early stopping is based on validation loss monitoring.

Table 3: Performance of three models on four benchmark datasets. Underlined values denote the baseline results. Values in parentheses indicate the relative change (%) with respect to the corresponding baseline.

Datasets	Metrics	BoostGCN	LightGCN	EA+LightGCN
ML-100K	HR@10	0.0692 (12.89%)	<u>0.0613</u>	0.0515 (-17.46%)
	MRR@10	0.0212 (7.61%)	<u>0.0197</u>	0.0166 (-15.74%)
	nDCG@10	0.0322 (10.27%)	<u>0.0292</u>	0.0246 (-16.44%)
	λ_2	0.3461 (0.55%)	<u>0.3442</u>	0.4921 (42.96%)
ML-1M	HR@10	0.0451 (23.90%)	<u>0.0364</u>	0.0365 (0.27%)
	MRR@10	0.0138 (21.05%)	<u>0.0114</u>	0.0114 (0.00%)
	nDCG@10	0.0210 (22.81%)	<u>0.0171</u>	0.0171 (0.00%)
	λ_2	0.4851 (0.17%)	<u>0.4843</u>	0.5080 (4.90%)
Yelp2018	HR@10	0.0368 (81.28%)	<u>0.0203</u>	0.0196 (-3.45%)
	MRR@10	0.0121 (80.60%)	<u>0.0067</u>	0.0064 (-4.48%)
	nDCG@10	0.0178 (81.63%)	<u>0.0098</u>	0.0094 (-4.08%)
	λ_2	0.0189 (2.72%)	<u>0.0184</u>	0.0252 (36.96%)
CiteULike	HR@10	0.0587 (42.48%)	<u>0.0412</u>	0.0379 (-8.01%)
	MRR@10	0.0186 (34.78%)	<u>0.0138</u>	0.0133 (-3.62%)
	nDCG@10	0.0278 (38.31%)	<u>0.0201</u>	0.0190 (-5.47%)
	λ_2	0.0097 (5.43%)	<u>0.0092</u>	0.1082 (1076.09%)

4.3.2 Experiments on other LGFs. To examine the applicability of our approach beyond LightGCN, we further conduct experiments on NGCF [43], another representative low-pass graph filter. As shown in Table 4, integrating the graph booster into NGCF leads to consistent performance gains. Together with the results on LightGCN, these findings suggest that the graph booster is generally effective for models built upon low-pass filtering.

Table 4: Performance of NGCF with and without Graph Booster (GB).

Datasets	Metrics	Vanilla	W/ GB	Improve (%)
ML-100K	HR@10	0.0594	0.0650	9.43
	MRR@10	0.0178	0.0197	10.67
	nDCG@10	0.0273	0.0301	10.26
ML-1M	HR@10	0.0395	0.0411	4.05
	MRR@10	0.0120	0.0125	5.83
	nDCG@10	0.0183	0.0190	4.37
Yelp2018	HR@10	0.0263	0.0304	15.59
	MRR@10	0.0087	0.0099	13.79
	nDCG@10	0.0128	0.0146	14.06
CiteULike	HR@10	0.0337	0.0420	24.63
	MRR@10	0.0104	0.0134	28.85
	nDCG@10	0.0157	0.0200	27.39

5 Related Work

5.1 Collaborative Filtering

Recommender systems are designed to identify items that users are likely to prefer based on historical user-item interactions. A widely adopted approach in this domain is Collaborative Filtering (CF), which generates recommendations by leveraging similarities among users or items [34, 35]. Typically, CF models parameterize users and items by learning latent representations directly from interaction data. In particular, Matrix Factorization (MF) has emerged as an effective CF-based technique, as it encodes dense latent representations in a low-dimensional space, thereby capturing intrinsic similarities between users and items [24, 33]. Concurrently, the success of deep neural networks has spurred significant efforts to enhance the interaction functions used for recommendation prediction [17, 20, 39, 48]. For example, NCF [20] synergistically combines MF with multi-layer perceptrons to introduce nonlinearity and flexibility into the interaction function. Additionally, methods such as

SVD++ [23], NAIS [19], and ACF [6] incorporate users' interaction histories through various pooling strategies, further advancing the learning of user and item representations.

5.2 Graph-based Collaborative Filtering

Recent years have witnessed a surge in the application of graph neural networks (GNNs) across various fields due to their strong expressive power [15, 22, 40, 49]. In recommender systems, user-item interactions naturally form a graph structure, where GNNs propagate information across edges so that connected nodes learn similar representations—capturing user preferences for different items [2, 12, 52]. Numerous models [18, 26, 28, 43] have been proposed to capture high-order connectivity on the user-item bipartite graph. Building on the insight that GCNs can be viewed as low-pass filters [46], researchers have leveraged graph signal processing (GSP) to design spectral-based models. For example, LCFN [55] and its subsequent refinement LGCN [56] apply spectral filtering to reduce high-frequency noise and better integrate user and item features, while methods like GDE [32] and SGFCF [31] further enhance the extraction of key graph features and address challenges related to data density and expressive power. Additionally, recent work has integrated self-supervised learning to improve robustness by enforcing consistency between representations and their augmented views [4, 47, 51, 53, 54]. More recently, ODE-based models [7, 8, 50] have been introduced to capture the continuous dynamics of node representations by formulating the propagation process as an ordinary differential equation. These Neural ODE-based GNNs offer adaptive computation and greater flexibility in controlling information diffusion, thereby mitigating over-smoothing and further boosting performance. However, existing methods have yet to explore the impact of increasing a graph's algebraic connectivity on the indirect enhancement of graph convolutional module performance. Furthermore, there is no proposed method for estimating algebraic connectivity subject to multi-element perturbation.

6 Conclusion

In this paper, we introduce BoostGCN, a novel model that enhances graph convolution capabilities for collaborative filtering. Unlike traditional models, which focus on designing complex graph convolution modules, BoostGCN reweights graph structures to enhance the classic graph convolution operation. Our experiments show that LGF in LightGCN perform well on graphs with high algebraic connectivity but struggles with low-connectivity graphs. Based on this, we propose a graph booster module that uses graph reweighting techniques, guided by an algebraic connectivity estimation method based on matrix perturbation theory, to improve connectivity. Extensive experiments on four benchmark datasets demonstrate that BoostGCN outperforms state-of-the-art methods, offering substantial gains in recommendation accuracy.

Acknowledgments

This work was supported by the Science and Technology Development Fund, Macau SAR (0126/2024/RIA2, 001/2024/SKL), CG-2026, GDST (2020B1212030003, 2023A0505030013), Nansha District (2023ZD001), MYRG-GRG2023-00186-FST-UMDF, MYRG-GRG2025-00234-FST.

References

- [1] Adrián Arnaiz-Rodríguez, Ahmed Begga, Francisco Escolano, and Nuria Oliver. 2022. DiffWire: Inductive Graph Rewiring via the Lovász Bound. In *Learning on Graphs Conference, LoG 2022, 9-12 December 2022, Virtual Event*, Vol. 198. PMLR, 15. <https://proceedings.mlr.press/v198/arnaiz-rodriguez22a.html>
- [2] Rianne van den Berg, Thomas N Kipf, and Max Welling. 2017. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263* (2017).
- [3] Aleksandar Bojchevski and Stephan Günnemann. 2019. Adversarial Attacks on Node Embeddings via Graph Poisoning. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 695–704. <http://proceedings.mlr.press/v97/bojchevski19a.html>
- [4] Xuheng Cai, Chao Huang, Lianghao Xia, and Xubin Ren. 2023. LightGCL: Simple Yet Effective Graph Contrastive Learning for Recommendation. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=FKXVK9dyMM>
- [5] Ashok K. Chandra, Prabhakar Raghavan, Walter L. Ruzzo, Roman Smolensky, and Prasoon Tiwari. 1989. The Electrical Resistance of a Graph Captures its Commute and Cover Times (Detailed Abstract). In *Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA*, David S. Johnson (Ed.). ACM, 574–586. doi:10.1145/73007.73062
- [6] Jingyuan Chen, Hanwang Zhang, Xiangnan He, Liqiang Nie, Wei Liu, and Tat-Seng Chua. 2017. Attentive Collaborative Filtering: Multimedia Recommendation with Item- and Component-Level Attention. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (Shinjuku, Tokyo, Japan) (SIGIR '17)*. Association for Computing Machinery, New York, NY, USA, 335–344. doi:10.1145/3077136.3080797
- [7] Jeongwhan Choi, Seoyoung Hong, Noseong Park, and Sung-Bae Cho. 2023. Blurring-Sharpener Process Models for Collaborative Filtering. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (Taipei, Taiwan) (SIGIR '23)*. Association for Computing Machinery, New York, NY, USA, 1096–1106. doi:10.1145/3539618.3591645
- [8] Jeongwhan Choi, Jinsung Jeon, and Noseong Park. 2021. LT-OCF: Learnable-Time ODE-based Collaborative Filtering. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (Virtual Event, Queensland, Australia) (CIKM '21)*. Association for Computing Machinery, New York, NY, USA, 251–260. doi:10.1145/3459637.3482449
- [9] Fan Chung, Linyuan Lu, and Van Vu. 2003. Spectra of random graphs with given expected degrees. *Proceedings of the National Academy of Sciences* 100, 11 (2003), 6313–6318. doi:10.1073/pnas.0937490100
- [10] Fan RK Chung. 1997. *Spectral graph theory*. Vol. 92. American Mathematical Soc.
- [11] Peter G Doyle and J Laurie Snell. 1984. *Random walks and electric networks*. Vol. 22. American Mathematical Soc.
- [12] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *The World Wide Web Conference (WWW '19)*. Association for Computing Machinery, New York, NY, USA, 417–426. doi:10.1145/3308558.3313488
- [13] Miroslav Fiedler. 1973. Algebraic connectivity of graphs. *Czechoslovak mathematical journal* 23, 2 (1973), 298–305.
- [14] Aric Hagberg, Pieter J Swart, and Daniel A Schult. 2007. *Exploring network structure, dynamics, and function using NetworkX*. Technical Report. Los Alamos National Laboratory (LANL).
- [15] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (Long Beach, California, USA) (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 1025–1035.
- [16] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (Dec. 2015), 19 pages. doi:10.1145/2827872
- [17] Xiangnan He and Tat-Seng Chua. 2017. Neural Factorization Machines for Sparse Predictive Analytics. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (Shinjuku, Tokyo, Japan) (SIGIR '17)*. Association for Computing Machinery, New York, NY, USA, 355–364. doi:10.1145/3077136.3080777
- [18] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, YongDong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (Virtual Event, China) (SIGIR '20)*. Association for Computing Machinery, New York, NY, USA, 639–648. doi:10.1145/3397271.3401063
- [19] Xiangnan He, Zhankui He, Jingkuan Song, Zhenguang Liu, Yu-Gang Jiang, and Tat-Seng Chua. 2018. NAIS: Neural Attentive Item Similarity Model for Recommendation. *IEEE Trans. on Knowl. and Data Eng.* 30, 12 (Dec. 2018), 2354–2366. doi:10.1109/TKDE.2018.2831682
- [20] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*. 173–182.
- [21] Adarsh Jamadandi, Celia Rubio-Madrigal, and Rebekka Burkholz. 2024. Spectral graph pruning against over-squashing and over-smoothing. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Article 331, 32 pages.
- [22] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SJU4ayYgl>
- [23] Yehuda Koren. 2008. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. 426–434.
- [24] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37. doi:10.1109/MC.2009.263
- [25] James R. Lee, Shayan Oveis Gharan, and Luca Trevisan. 2014. Multiway Spectral Partitioning and Higher-Order Cheeger Inequalities. *J. ACM* 61, 6, Article 37 (Dec. 2014), 30 pages. doi:10.1145/2665063
- [26] Fan Liu, Zhiyong Cheng, Lei Zhu, Zan Gao, and Liqiang Nie. 2021. Interest-aware message-passing GCN for recommendation. In *Proceedings of the web conference 2021*. 1296–1305.
- [27] Jiahao Liu, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2023. Personalized graph signal processing for collaborative filtering. In *Proceedings of the ACM Web Conference 2023*. 1264–1272.
- [28] Kelong Mao, Jieming Zhu, Xi Xiao, Biao Lu, Zhaowei Wang, and Xiuqiang He. 2021. UltraGCN: ultra simplification of graph convolutional networks for recommendation. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 1253–1262.
- [29] Maximilian Nickel and Douwe Kiela. 2017. Poincaré embeddings for learning hierarchical representations. *Advances in neural information processing systems* 30 (2017).
- [30] Antonio Ortega, Pascal Frossard, Jelena Kovačević, José MF Moura, and Pierre Vanderghenst. 2018. Graph signal processing: Overview, challenges, and applications. *Proc. IEEE* 106, 5 (2018), 808–828.
- [31] Shaowen Peng, Xin Liu, Kazunari Sugiyama, and Tsunenori Mine. 2024. How powerful is graph filtering for recommendation. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 2388–2399.
- [32] Shaowen Peng, Kazunari Sugiyama, and Tsunenori Mine. 2022. Less is more: Reweighting important spectral graph features for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 1273–1282.
- [33] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*. 452–461.
- [34] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. Grouplens: An open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM conference on Computer supported cooperative work*. 175–186.
- [35] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*. 285–295.
- [36] Yifei Shen, Yongji Wu, Yao Zhang, Caihua Shan, Jun Zhang, B. Khaled Letiaief, and Dongsheng Li. 2021. How Powerful is Graph Convolution for Recommendation?. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (Virtual Event, Queensland, Australia) (CIKM '21)*. Association for Computing Machinery, New York, NY, USA, 1619–1629. doi:10.1145/3459637.3482264
- [37] Joseph Sill. 1997. Monotonic networks. In *Proceedings of the 11th International Conference on Neural Information Processing Systems (Denver, CO) (NIPS'97)*. MIT Press, Cambridge, MA, USA, 661–667.
- [38] G.W. Stewart and J. Sun. 1990. *Matrix Perturbation Theory*. Elsevier Science. <https://books.google.com/books?id=bIYEogEACAAJ>
- [39] Yi Tay, Luu Anh Tuan, and Siu Cheung Hui. 2018. Latent relational metric learning via memory-based attention for collaborative ranking. In *Proceedings of the 2018 world wide web conference*. 729–739.
- [40] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rJXmpikCZ>
- [41] Hao Wang, Binyi Chen, and Wu-Jun Li. 2013. Collaborative topic regression with social regularization for tag recommendation. In *IJCAI*, Vol. 13. 2719–2725.
- [42] Hao Wang, Defu Lian, Hanghang Tong, Qi Liu, Zhenya Huang, and Enhong Chen. 2021. Hypersorec: Exploiting hyperbolic user and item representations with multiple aspects for social-aware recommendation. *ACM Transactions on Information Systems (TOIS)* 40, 2 (2021), 1–28.
- [43] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural Graph Collaborative Filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*

- (Paris, France) (SIGIR'19). Association for Computing Machinery, New York, NY, USA, 165–174. doi:10.1145/3331184.3331267
- [44] Eugene P Wigner. 1958. On the distribution of the roots of certain symmetric matrices. *Annals of Mathematics* 67, 2 (1958), 325–327.
- [45] Eugene P Wigner. 1993. Characteristic vectors of bordered matrices with infinite dimensions i. In *The Collected Works of Eugene Paul Wigner: Part A: The Scientific Papers*. Springer, 524–540.
- [46] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying Graph Convolutional Networks. In *Proceedings of the 36th International Conference on Machine Learning*, Vol. 97. PMLR, 6861–6871. <https://proceedings.mlr.press/v97/wu19e.html>
- [47] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. 2021. Self-supervised graph learning for recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 726–735.
- [48] Yao Wu, Christopher DuBois, Alice X Zheng, and Martin Ester. 2016. Collaborative denoising auto-encoders for top-n recommender systems. In *Proceedings of the ninth ACM international conference on web search and data mining*. 153–162.
- [49] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=ryGs6iA5Km>
- [50] Ke Xu, Weizhi Zhang, Zhihe Song, Yuanjie Zhu, and Philip S Yu. 2025. Graph Neural Controlled Differential Equations For Collaborative Filtering. In *Companion Proceedings of the ACM on Web Conference 2025*. 1446–1449.
- [51] Tiansheng Yao, Xinyang Yi, Derek Zhiyuan Cheng, Felix Yu, Ting Chen, Aditya Menon, Lichan Hong, Ed H Chi, Steve Tjoa, Jieqi Kang, et al. 2021. Self-supervised learning for large-scale item recommendations. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 4321–4330.
- [52] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
- [53] Junliang Yu, Xin Xia, Tong Chen, Lizhen Cui, Nguyen Quoc Viet Hung, and Hongzhi Yin. 2023. XSimGCL: Towards extremely simple graph contrastive learning for recommendation. *IEEE Transactions on Knowledge and Data Engineering* 36, 2 (2023), 913–926.
- [54] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Lizhen Cui, and Quoc Viet Hung Nguyen. 2022. Are graph augmentations necessary? simple graph contrastive learning for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 1294–1303.
- [55] Wenhui Yu and Zheng Qin. 2020. Graph convolutional network for recommendation with low-pass collaborative filters. In *International conference on machine learning*. PMLR, 10936–10945.
- [56] Wenhui Yu, Zixin Zhang, and Zheng Qin. 2022. Low-Pass Graph Convolutional Network for Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 8 (Jun. 2022), 8954–8961. doi:10.1609/aaai.v36i8.20878
- [57] Wentao Zhang, Mingyu Yang, Zeang Sheng, Yang Li, Wen Ouyang, Yangyu Tao, Zhi Yang, and Bin Cui. 2021. Node dependent local smoothing for scalable graph learning. *Advances in Neural Information Processing Systems* 34 (2021), 20321–20332.

7 Appendix

7.1 Monotonicity Analysis of the Spectral Filter in LightGCN

We consider $f(\lambda) = \frac{1}{K+1} \sum_{l=0}^K (1-\lambda)^l$, $K \in \mathbb{Z}_{\geq 0}$, $\lambda \in [0, 2]$. By the geometric series identity, for $\lambda \neq 0$, $f(\lambda) = \frac{1-(1-\lambda)^{K+1}}{\lambda(K+1)}$, and by continuity $f(0) = 1$.

To analyze monotonicity, we get the derivative:

$$f'(\lambda) = \frac{(K+1)\lambda(1-\lambda)^K - 1 + (1-\lambda)^{K+1}}{\lambda^2(K+1)}. \quad (23)$$

Defining $g(\lambda) = (1-\lambda)^K(1+K\lambda)$, this simplifies to:

$$f'(\lambda) = \frac{g(\lambda) - 1}{\lambda^2(K+1)}. \quad (24)$$

Since the denominator is positive for $\lambda > 0$, the sign of $f'(\lambda)$ is determined by $g(\lambda) - 1$.

On $\lambda \in [0, 1]$, we have $g(0) = 1$ and

$$\ln g(\lambda) = K \ln(1-\lambda) + \ln(1+K\lambda), \quad (25)$$

with derivative

$$\frac{d}{d\lambda} \ln g(\lambda) = -\frac{K}{1-\lambda} + \frac{K}{1+K\lambda} < 0. \quad (26)$$

Hence $g(\lambda)$ is strictly decreasing, $g(\lambda) < 1$ for $\lambda \in (0, 1)$, and therefore $f'(\lambda) < 0$. Thus $f(\lambda)$ is strictly decreasing on $[0, 1]$.

On $\lambda \in [1, 2]$, the behavior depends on the parity of K . If K is odd, $(1-\lambda)^K < 0$ and $m(\lambda) < 0$, so $f'(\lambda) < 0$ and $f(\lambda)$ is strictly decreasing. If K is even, then $(1-\lambda)^K = (\lambda-1)^K > 0$, and

$$g'(\lambda) = K(\lambda-1)^{K-1}\lambda(K+1) > 0 \quad (\lambda > 1). \quad (27)$$

Thus $g(\lambda)$ increases from $g(1) = 0$ to $g(2) = 1 + 2K > 1$. By the intermediate value theorem, there exists a unique $\lambda^* \in (1, 2)$ with $g(\lambda^*) = 1$. Therefore $f'(\lambda) < 0$ for $\lambda \in (1, \lambda^*)$ and $f'(\lambda) > 0$ for $\lambda \in (\lambda^*, 2]$.

In summary, $f(\lambda)$ is strictly decreasing on $\lambda \in [0, 1]$. On $\lambda \in [1, 2]$, it is decreasing for odd K , and for even K it decreases on $(1, \lambda^*)$ and increases on $(\lambda^*, 2]$, where λ^* solves $(\lambda-1)^K(1+K\lambda) = 1$.

7.2 Spectral Limitation of Mid-Frequency Components in Graph Convolution

In collaborative filtering, graph convolution applies spectral filters to graph signals. We identify a theoretical limitation: components corresponding to zero eigenvalues of the adjacency matrix, where $\lambda_i = 1 - \mu_i \approx 1$, cannot distinguish users with different interaction patterns. The following derivation formalizes this limitation.

LEMMA 2.2. *Let $\tilde{\mathbf{L}}$ denote the symmetric normalized Laplacian matrix of a user-item bipartite graph. After l layers of graph convolution, the squared Euclidean distance between the embeddings of two arbitrary users u and v is given by:*

$$\|\mathbf{e}_u^{(l)} - \mathbf{e}_v^{(l)}\|^2 = \sum_{i=1}^n (\alpha_{u,i} - \alpha_{v,i})^2 (1 - \lambda_i)^{2l},$$

where λ_i is the i -th eigenvalue of $\tilde{\mathbf{L}}$, and $\alpha_{u,i}$, $\alpha_{v,i}$ are the projection coefficients of the embeddings $\mathbf{e}_u^{(0)}$ and $\mathbf{e}_v^{(0)}$ onto the eigenvector $\tilde{\mathbf{v}}_i$.

PROOF. We start with the spectral decomposition of the normalized adjacency matrix. Since $\tilde{\mathbf{A}}$ is symmetric, it admits the eigendecomposition: $\tilde{\mathbf{A}} = \tilde{\mathbf{V}}^T \mathbf{M} \tilde{\mathbf{V}}$, $\mathbf{M} = \text{diag}(\mu_1, \dots, \mu_n)$, where $\tilde{\mathbf{V}} = [\tilde{\mathbf{v}}_1, \dots, \tilde{\mathbf{v}}_n]$ denotes the orthonormal eigenvectors of $\tilde{\mathbf{A}}$ and μ_i are the corresponding eigenvalues.

Next, the initial embedding matrix $\mathbf{E}$ is projected onto the spectral basis, yielding: $\mathbf{E} = \sum_{i=1}^n \tilde{\mathbf{v}}_i \alpha_i^T$, where $\alpha_i = \mathbf{E}^T \tilde{\mathbf{v}}_i \in \mathbb{R}^d$. After l layers of propagation, the embeddings are updated via repeated application of $\tilde{\mathbf{A}}$: $\mathbf{E}^{(l)} = \tilde{\mathbf{A}}^l \mathbf{E} = \sum_{i=1}^n \mu_i^l \tilde{\mathbf{v}}_i \alpha_i^T$.

Hence, the embedding of user u becomes:

$$\mathbf{e}_u^{(l)} = \sum_{i=1}^n \mu_i^l \alpha_{u,i} \tilde{\mathbf{v}}_i, \quad \text{where } \alpha_{u,i} = \tilde{\mathbf{v}}_i^T \mathbf{e}_u. \quad (28)$$

To measure user-level discrepancy, we compute the squared Euclidean distance between two users u and v :

$$\begin{aligned} \|\mathbf{e}_u^{(l)} - \mathbf{e}_v^{(l)}\|^2 &= \left\| \sum_{i=1}^n \mu_i^l (\alpha_{u,i} - \alpha_{v,i}) \tilde{\mathbf{v}}_i \right\|^2 \\ &= \sum_{i=1}^n (\alpha_{u,i} - \alpha_{v,i})^2 (1 - \lambda_i)^{2l}, \end{aligned} \quad (29)$$

where the orthonormality of eigenvectors is used in the final step.

While the proof is illustrated from the view of users, the same spectral analysis and conclusions hold symmetrically for items. $\square$

We can directly get the Corollary 2.2.1, the mid-frequency components (those with $\lambda_i \approx 1$) vanish due to propagation: $\sum_{i: \lambda_i \approx 1} (\alpha_{u,i} - \alpha_{v,i})^2 \cdot (1 - \lambda_i)^{2l} \approx 0$, for $l \geq 1$. This result confirms that mid-frequency components inherently fail to distinguish users, regardless of their interaction differences.

7.3 Proof of Spectral Concentration Theorem

Motivated by our Empirical Observation (2) that λ_2 consistently reflects the informative signals ratio, we now provide a theoretical analysis to justify this relationship.

THEOREM 2.4 (SPECTRAL CONCENTRATION THEOREM). *Let $\mathcal{G}_A$ and $\mathcal{G}_B$ be two random bipartite graphs with a degree sequence satisfying $d_{\min} \gg \sqrt{\tilde{d}}$, where $\tilde{d}$ denotes the average vertex degree (i.e. $\tilde{d} = \frac{1}{|V|} \sum_{v \in V} d_v$). Then for any $\delta \in (0, 1)$, $\lambda_2(\mathcal{G}_A) > \lambda_2(\mathcal{G}_B) \Rightarrow R_\delta(\mathcal{G}_A) < R_\delta(\mathcal{G}_B)$.*

PROOF. We begin with some concepts. Let $\mathcal{G}$ be a user-item bipartite graph with symmetric normalized Laplacian $\tilde{L}$, and let $\{\lambda_n : 0 \leq n \leq N\}$ be its eigenvalues. To quantify spectral concentration, we have defined the **low-frequency signal ratio** as: $R_\delta(\mathcal{G}) = \frac{K}{N}$, where N is the dimension of the matrix, K is the largest index of the eigenvalue such that $\lambda_K < \delta < 1$.

User-item graphs in the real world often follow power-law degree distributions [29, 42], making it natural to approximate them as random scale-free networks. It is important to note that the application of related spectral theorems (e.g., the semicircle law [9, 44, 45]) is asymptotic and requires strong degree conditions. While our finite empirical graphs may not strictly satisfy these conditions, the theoretical framework provides valuable intuition. According to the work of Chung et al. [9], for random scale-free graphs with a sufficiently large minimum expected degree, they give the following semicircle law [44, 45]:

THEOREM 7.1 (THEOREM 6 OF [9]). *Let $W(x)$ denote the cumulative distribution function of the semicircle law with density $w(x) = \frac{2}{\pi} \sqrt{1 - x^2} \cdot \mathbf{1}_{[-1, 1]}(x)$. Define the normalized centered Laplacian as $C_{\text{nor}} = \frac{2}{\sqrt{\tilde{d}}} (\mathbf{I} - \tilde{L} - \mathbf{P}_0)$, where $\tilde{d}$ denotes the expected average degree and $\mathbf{P}_0$ is the orthogonal projector onto the principal eigenspace of $\tilde{L}$. Let $N(x)$ denote the number of eigenvalues of C_{nor} less than or equal to x , and define the empirical spectral distribution by $W_n(x) = \frac{1}{n} N(x)$. For random graphs whose degree sequences satisfy $d_{\min} \gg \sqrt{\tilde{d}}$, it has been shown that $W_n(x)$ converges in probability to $W(x)$ as $n \rightarrow \infty$.*

This theorem implies that, after radius normalization and removal of the trivial eigenvalue, the nontrivial eigenvalues of $\tilde{L}$ asymptotically follow the semicircle law. The distribution is centered at $x = 1$ with radius $r \approx \frac{\sqrt{\tilde{d}}}{2}$. As the algebraic connectivity is the smallest nontrivial eigenvalue, in this setting, the algebraic connectivity asymptotically satisfies $\lambda_2 \approx \mathbb{E}[1 - r]$. Hence, in the limit as the number of nodes $n \rightarrow \infty$, the following equivalence

holds with high probability:

$$\lambda_2(\mathcal{G}_A) > \lambda_2(\mathcal{G}_B) \iff r^{(A)} < r^{(B)}. \quad (30)$$

That is, larger algebraic connectivity implies a tighter spectral concentration around the center.

We now can connect the low-frequency signal ratio with the semicircle law:

$$R_\delta(\mathcal{G}) \triangleq W_n(\delta) = \int_{-1}^{\frac{\delta-1}{r}} \frac{2}{\pi} \sqrt{1 - t^2} dt. \quad (31)$$

To derive a closed-form expression, we apply the substitution $t = -\sin \theta$, where $\theta \in [-\frac{\pi}{2}, \frac{\pi}{2}]$, yielding:

$$R_\delta(\mathcal{G}) = \frac{1}{\pi} \left(\arcsin\left(\frac{\delta-1}{r}\right) + \frac{\delta-1}{r} \sqrt{1 - \left(\frac{\delta-1}{r}\right)^2} + \frac{\pi}{2} \right). \quad (32)$$

The partial derivative of $R_\delta(\mathcal{G})$ with respect to r is:

$$\begin{aligned} \frac{\partial R}{\partial r} &= \frac{1}{\pi} \frac{\partial}{\partial r} \arcsin\left(\frac{\delta-1}{r}\right) + \frac{1}{\pi} \frac{\partial}{\partial r} \left(\frac{\delta-1}{r} \sqrt{1 - \left(\frac{\delta-1}{r}\right)^2} \right) \\ &= \frac{1}{\pi} \frac{1}{\sqrt{1 - \left(\frac{\delta-1}{r}\right)^2}} \left(-\frac{\delta-1}{r^2} \right) - \frac{\delta-1}{r^2} \cdot \frac{1 - 2\left(\frac{\delta-1}{r}\right)^2}{\sqrt{1 - \left(\frac{\delta-1}{r}\right)^2}} \\ &= -\frac{2(\delta-1)\sqrt{r^2 - (\delta-1)^2}}{\pi r^3} \end{aligned} \quad (33)$$

Since $0 < 1 - \delta < r$ and $\frac{\partial R_\delta(\mathcal{G})}{\partial r} > 0$, this function is strictly increasing in r . Thus, for fixed $\delta \in (0, 1)$ and $r^{(A)} < r^{(B)}$, we conclude:

$$R_\delta(\mathcal{G}_A) < R_\delta(\mathcal{G}_B). \quad (34)$$

$\square$

7.4 Perturbation Analysis of the Normalized Laplacian

Let Δ_A denote a small perturbation to the adjacency matrix A , inducing a change in the degree matrix $\Delta_D = \text{diag}(\sum_j \Delta_A[i, j])$. Let $\tilde{v}_i$ denote a unit-norm eigenvector of $\tilde{L}$ corresponding to eigenvalue λ_i . Rayleigh quotient for the normalized Laplacian $\tilde{L}$ is $\lambda_i = \tilde{v}_i^\top \tilde{L} \tilde{v}_i$. According to standard *matrix perturbation theory* [38], for small perturbation $\Delta_{\tilde{L}} = \tilde{L}' - \tilde{L}$, first-order perturbation is given by:

$$\Delta \lambda_i \approx \tilde{v}_i^\top \Delta_{\tilde{L}} \tilde{v}_i \quad (35)$$

$$\stackrel{(i)}{=} \tilde{v}_i^\top \left(\underbrace{\tilde{L} - (\mathbf{D} + \Delta_D)^{-\frac{1}{2}} (\mathbf{A} + \Delta_A) (\mathbf{D} + \Delta_D)^{-\frac{1}{2}}}_{\text{perturbed Laplacian}} - \underbrace{\tilde{L} + \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}}_{\text{original Laplacian}} \right) \tilde{v}_i \quad (37)$$

$$\stackrel{(ii)}{\approx} \tilde{v}_i^\top \left(\underbrace{\frac{1}{2} \mathbf{D}^{-\frac{3}{2}} \Delta_D \mathbf{A} \mathbf{D}^{-\frac{1}{2}} + \frac{1}{2} \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{3}{2}} \Delta_D}_{\text{indirect perturbation}} - \underbrace{\mathbf{D}^{-\frac{1}{2}} \Delta_A \mathbf{D}^{-\frac{1}{2}}}_{\text{direct perturbation}} \right) \tilde{v}_i \quad (38)$$

$$\stackrel{(iii)}{=} \tilde{v}_i^\top \left((\mathbf{I} - \tilde{L}) \mathbf{D}^{-1} \Delta_D - \mathbf{D}^{-\frac{1}{2}} \Delta_A \mathbf{D}^{-\frac{1}{2}} \right) \tilde{v}_i \quad (39)$$

$$\stackrel{(iv)}{=} (1 - \lambda_i) \tilde{v}_i^\top \mathbf{D}^{-1} \Delta_D \tilde{v}_i - \tilde{v}_i^\top \mathbf{D}^{-\frac{1}{2}} \Delta_A \mathbf{D}^{-\frac{1}{2}} \tilde{v}_i \quad (40)$$

Explanation of steps: (i) Substitutes the definition of the perturbed Laplacian and expands the difference with the original Laplacian. (ii) Uses the first-order Taylor expansion of $(\mathbf{D} + \Delta_{\mathbf{D}})^{-\frac{1}{2}} = \mathbf{D}^{-\frac{1}{2}} - \frac{1}{2}\mathbf{D}^{-\frac{3}{2}}\Delta_{\mathbf{D}} + O(\|\Delta_{\mathbf{D}}\|^2)$ to isolate perturbation terms. (iii) Applies the identity $\mathbf{A} = \mathbf{D}^{1/2}(\mathbf{I} - \tilde{\mathbf{L}})\mathbf{D}^{1/2}$ to simplify. (iv) Uses the eigenrelation $\tilde{\mathbf{L}}\tilde{\mathbf{v}}_i = \lambda_i\tilde{\mathbf{v}}_i$ to compute the final expression.

Finally, by expanding the matrix form into element-wise notation and using the Kronecker delta δ_{jk} and the definition of $\Delta_{\mathbf{D}}$, we arrive at:

$$\Delta\lambda_i \approx \sum_{j,k} \left(\frac{\tilde{v}_i[j]\tilde{v}_i[k]}{\sqrt{\mathbf{D}_{jj}\mathbf{D}_{kk}}} - \lambda_i \frac{\tilde{v}_i[j]^2}{\mathbf{D}_{jj}} \delta_{jk} \right) \Delta_{A_{jk}}. \quad (41)$$

The final expression quantifies the sensitivity of eigenvalues to local structural changes, capturing both the direct influence of perturbations in adjacency and the indirect adjustments via degree variation, each modulated by the spectral profile of $\tilde{\mathbf{L}}$.

7.5 Does the $\Delta\lambda_2$ Regularizer Really Work?

We conducted ablation studies to validate the effectiveness of the proposed regularization term $\Delta\lambda_2$ and assess its impact on model performance. In these experiments, we set the coefficient γ_2 to 0, eliminating its influence on graph reweighting. This setup examined whether the absence of regularization guidance impacted performance. Table 5 presents the results, summarizing performance metrics across different conditions. Generally, removing the regularizer led to a performance decline, highlighting its role in guiding the reweighting process toward optimal solutions.

Table 5: Ablation study on $\Delta\lambda_2$ penalty. W/o $\Delta\lambda_2$ refers to the variant where $\Delta\lambda_2$ is removed from the vanilla model.

Datasets	Metrics	Vanilla	W/o $\Delta\lambda_2$	Drop (%)
ML-100K	HR@10	0.0692	0.0676	-2.31
	MRR@10	0.0212	0.0211	-0.47
	nDCG@10	0.0322	0.0318	-1.24
ML-1M	HR@10	0.0451	0.0450	-0.22
	MRR@10	0.0138	0.0137	-0.72
	nDCG@10	0.0210	0.0207	-1.43
Yelp2018	HR@10	0.0368	0.0357	-2.99
	MRR@10	0.0121	0.0117	-3.31
	nDCG@10	0.0178	0.0172	-3.37
CiteULike	HR@10	0.0587	0.0586	-0.17
	MRR@10	0.0186	0.0186	0.00
	nDCG@10	0.0278	0.0278	0.00

7.6 Does the Approximation Formula for $\Delta\lambda_2$ Accurately Estimate the Change in λ_2 ?

To assess the effectiveness of the proposed approximation equation for estimating algebraic connectivity, we conducted experiments on an Erdős-Rényi (ER) graph with parameters $(|\mathcal{V}|, |\mathcal{E}|) = (30, 211)^7$, as illustrated in Figure 3. At each iteration, we introduced random perturbations to the adjacency matrix and used the true algebraic connectivity (computed via eigen-decomposition) as a reference to assess the accuracy of the proposed approximation. The results

⁷We used the *NetworkX* [14] package to generate an ER graph with 30 nodes, setting the edge creation probability to 0.5.

demonstrate that the approximation equation in Eq. 16 consistently provides a reliable estimate of algebraic connectivity.

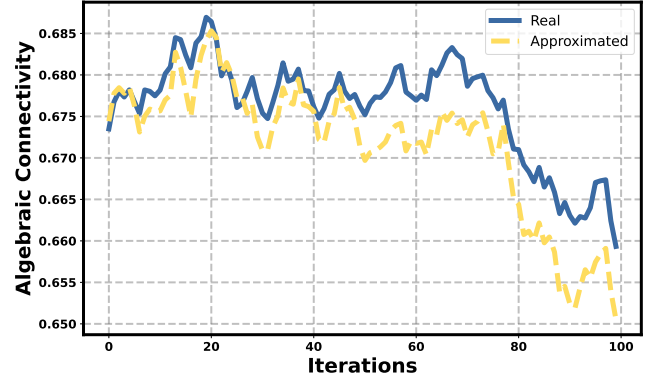


Figure 3: The variations in true algebraic connectivity of a random ER graph with 30 nodes and 211 edges alongside the corresponding changes estimated using our approximation method. The results suggest that the discrepancy between the approximated and true algebraic connectivity remains within an acceptable range.

7.7 Hyperparameter Analysis

We investigate the impact of varying the number of convolutional layers in our graph model. Specifically, we assess performance across configurations with 1 to 5 layers, as depicted in Figure 4. The results indicate that performance peaks at 2 or 3 layers for ML series and Yelp2018 datasets, while the CiteULike datasets exhibit consistent improvement, reaching their optimal performance at 4 or 5 layers. These findings align with our expectations, as the CiteULike dataset is highly sparse, implying that additional layers effectively capture meaningful multi-hop information.

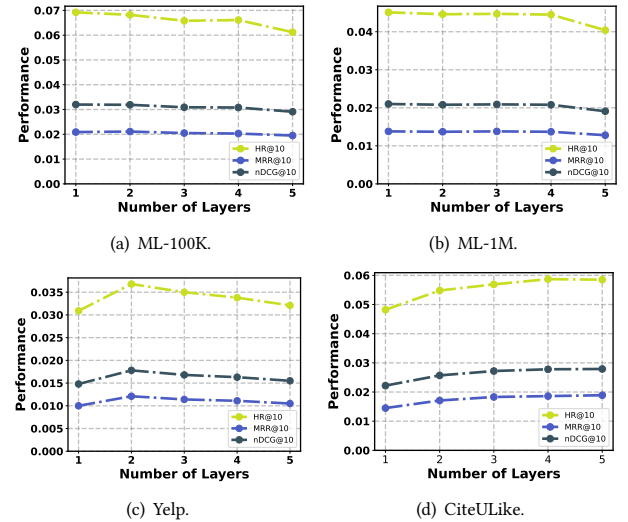


Figure 4: BoostGCN performance with different number of layers L .